System Design and Architecture Specification

Distributed MLOps Platform

Technical Report & Engineering Decisions

Team 3

April 28, 2026

Contents

1. Introduction & Problem Statement

The transition of machine learning models from isolated experimental environments (e.g., Jupyter Notebooks) to highly available, reliable production systems remains one of the most significant bottlenecks in modern data engineering [1]. A core challenge lies in bridging the gap between stateful, computationally heavy training processes and stateless, low-latency inference requirements [2].

Without a unified operational framework, organizations face fractured data pipelines, loss of model traceability, and severe scalability limits. These limits are largely dictated by the inherent constraints of Python-based execution environments–specifically, the Global Interpreter Lock (GIL) [3] and memory fragmentation during heavy matrix operations.

To solve this, we engineered a **Distributed MLOps Platform**: a lightweight, highly decoupled infrastructure designed to manage the comprehensive lifecycle of machine learning models–from automated asynchronous training to mathematically optimized inference serving.

2. Methods

2.1. High-Level System Architecture

To eliminate the operational complexity of Kubernetes while still providing cgroup-based resource constraints (memory and CPU limits with reservations per service) and private networking via Docker bridge, the entire orchestration layer is built on Docker Compose. The platform comprises nine containerized services: PostgreSQL, MinIO, MLflow, Orchestrator, Training Worker, Inference Service, Monitoring Service, Monitoring Dashboard, and a MinIO seed initializer.

The architecture achieves low coupling through a hybrid communication topology, conceptually divided into three distinct operational planes:

1. **Control Plane:** The Orchestrator Service acts as the primary API facade, providing endpoints for dataset registration, training job creation, model listing, and model promotion. It manages system state directly via PostgreSQL.
2. **Execution Plane:** Handles computations, strictly divided into an asynchronous polling worker (Training) and an event-driven web service (Inference).
3. **Storage & Observability Plane:** Persists raw data, model binaries, metrics, and state via PostgreSQL, MinIO, and MLflow.

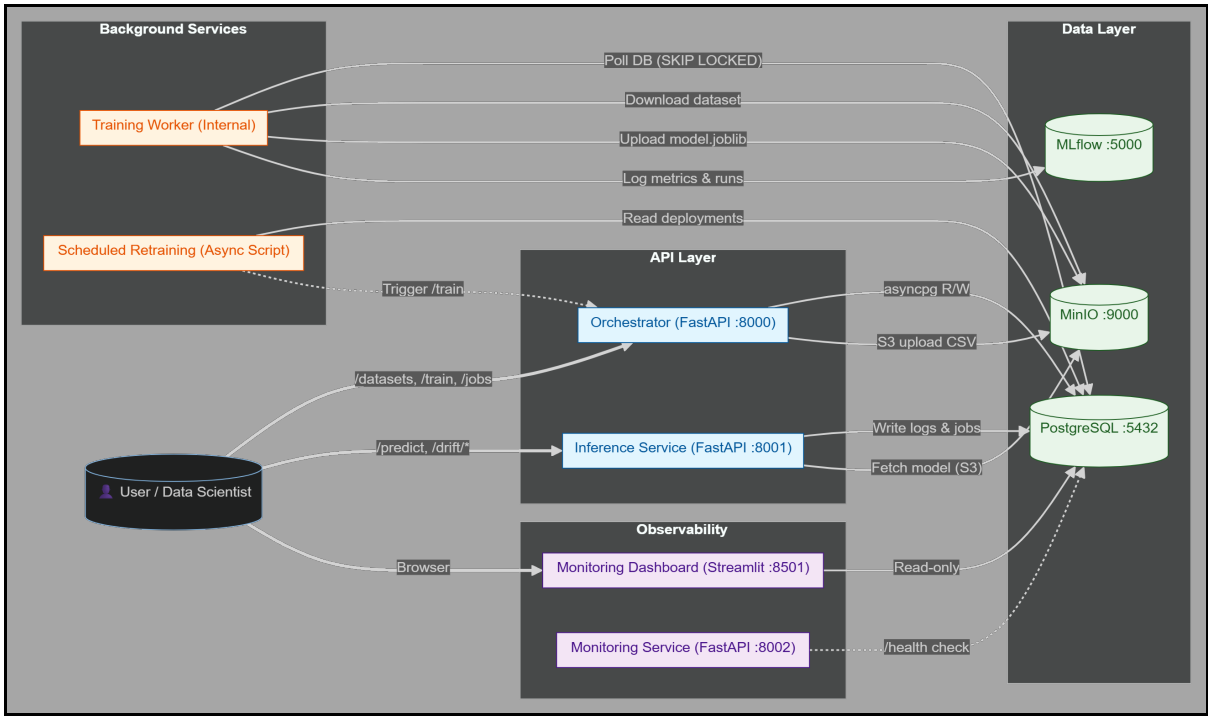

Figure 1: System Architecture Diagram showcasing the Control, Execution, and Storage planes.

The primary inter-service communication pattern is database-driven: the Training Worker polls PostgreSQL for pending jobs, and all state transitions are persisted as row mutations. This ensures fault tolerance: the Orchestrator can accept training requests even if zero workers are online; once booted, workers drain the backlog. One exception is the Scheduled Retraining Service, which communicates with the Orchestrator via REST (*HTTP POST* to `/train`). The Inference Service also creates retraining `jobs` by writing directly to the jobs table, bypassing the Orchestrator.

2.2. Model Lifecycle Workflow

To understand how the architectural planes interact, it is best to trace the lifecycle of a predictive model from raw data to live inference.

User Story Context: A Data Scientist requires a new predictive model based on updated business data. The scientist first uploads the dataset via `POST /datasets` (multipart CSV upload), which stores the file in MinIO and registers it in the `datasets` table. The scientist then issues a `POST /train` request referencing the dataset. The platform autonomously handles the rest.

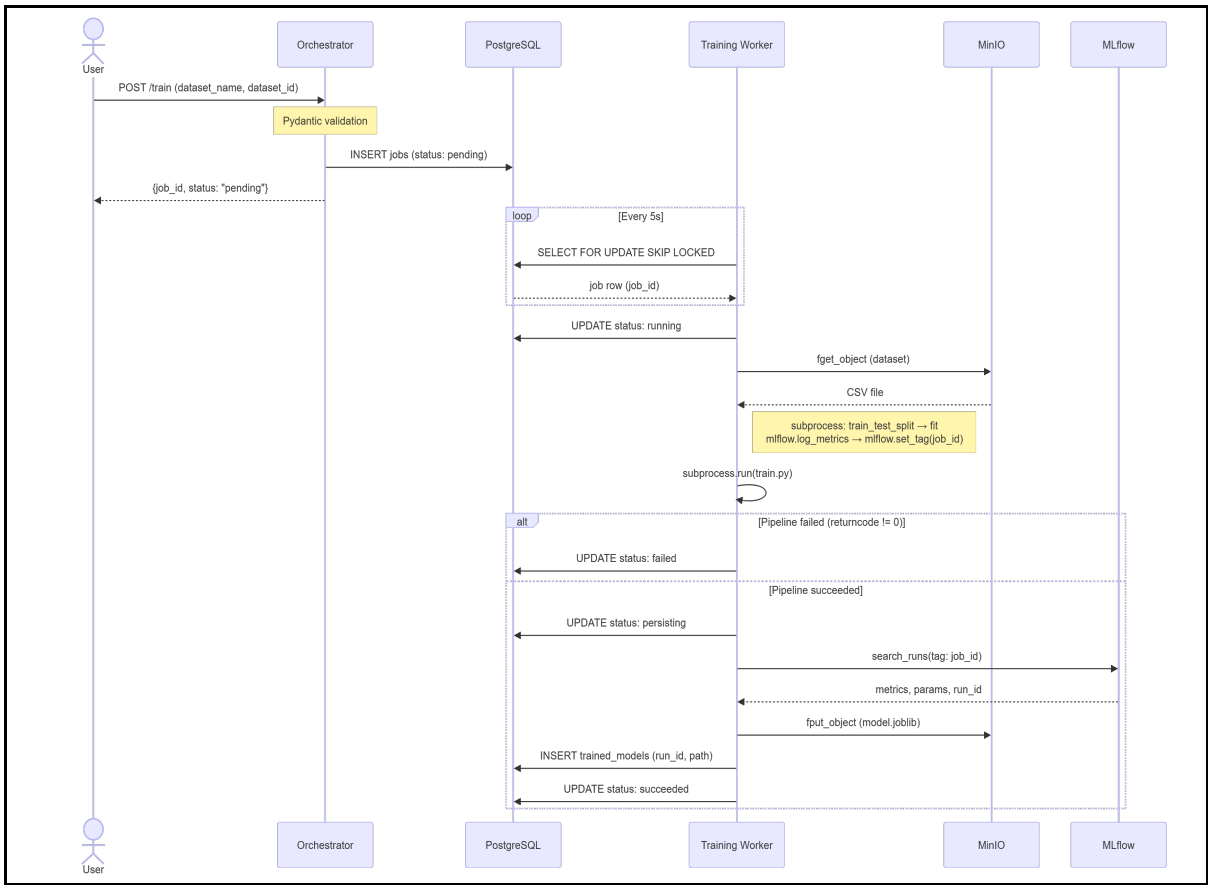

Figure 2: The Lifecycle of a Model.

As illustrated in Figure 2, the process follows a strict state machine:

1. **Registration (`Pending`):** The client submits a POST request to `/train` with query parameters (`dataset_name`, `dataset_id`, `dataset_path`). FastAPI performs automatic type validation on these parameters. The Orchestrator synchronously inserts a record into the `jobs` table with a `pending` status.
2. **Atomic Capture (`Running`):** The Training Worker, running as a background asyncio task within a FastAPI application, continuously polls the database. It atomically locks the row via `FOR UPDATE SKIP LOCKED` and updates the status to `running`.
3. **Isolated Execution:** The Worker pulls the dataset from MinIO to a temporary directory (configurable via `WORKER_TMP_DIR`), and executes the ML pipeline in an isolated OS process.
4. **Telemetry Linking:** During subprocess execution, the training script logs metrics and hyperparameters to MLflow and tags the run with the database `job_id`.

5. **Telemetry Completion:** After the subprocess finishes, the Worker queries MLflow for the run matching the tag `job_id`, retrieves metrics (Accuracy, Precision, Recall, F1) and hyperparameters, and extracts the reference profile for drift detection.
6. **Persistence (`Persisting`):** The Worker updates the job status to `persisting`, uploads the serialized model to MinIO via `fput_object` under the path {`model_name`}/{`model_version`}`.joblib`, and writes metadata to the `trained_models` table.
7. **Finalization (`Succeeded`):** The job status is updated to `succeeded`.

State Machine Resilience: If an unhandled exception occurs during the training subprocess execution, the Worker catches the non-zero exit code (checked via `result.returncode`) and transitions the database status to `failed`. If the Worker process itself is terminated (e.g., SIGKILL) while a job is in the `running` state, the next Worker startup resets all stuck `running` jobs back to `pending` via a single SQL `UPDATE` query, allowing them to be reprocessed. This prevents orphaned jobs from permanently blocking the queue.

2.3. Component Engineering Decisions

Each subsystem was engineered to solve specific bottlenecks in the MLOps pipeline. The following sections detail the rationale behind their technical implementations.

In addition to the three core components described below, the Orchestrator exposes endpoints for dataset upload (`POST /datasets`), model listing (`GET /models`), model promotion (`POST /promote`), and deployment management (`GET /deployments`). The Inference Service also features drift detection, prediction logging, and a Circuit Breaker for MinIO resilience. A Scheduled Retraining Service periodically checks active deployments and triggers retraining via the Orchestrator API.

2.3.1. Job Orchestration & State Management

To track job progression reliably, we required an ACID-compliant state machine. We selected PostgreSQL as the single source of truth for all service state. JSONB columns are used where flexible schema is needed – specifically `metrics` and `parameters` in the `trained_models` table, and `input_data` and `prediction` in the `prediction_logs` table. This eliminates the need for an external message broker like RabbitMQ or Kafka, keeping the platform lightweight.

However, horizontally scaling the Training Workers introduces race conditions where multiple workers might attempt to train the same `pending` job simultaneously. To solve this, we implemented our queue directly in PostgreSQL using the `FOR UPDATE SKIP LOCKED` **paradigm**. This guarantees absolute database-level atomicity: when a worker queries the table, it locks the row it intends to process and hides it from concurrent queries by other workers.

2.3.2. Asynchronous Training Environment

The Training Worker is a FastAPI application that exposes a `/health` endpoint for orchestration-level monitoring. The actual polling loop runs as an asynchronous background task (`asyncio.create_task`) within the FastAPI lifespan. For our mathematical baseline, we integrated a Scikit-Learn Pipeline composed of `SimpleImputer(strategy="median")`, `StandardScaler`, and `LogisticRegression (max_iter=1000, random_state=42)`. While strictly CPU-bound and mathematically simple, it validates our full artifact management and state-transition workflows.

A critical challenge with Scikit-Learn/Pandas is memory management. Executing memory-intensive pipelines directly within the worker's main Python process risks memory fragmentation and Out-Of-Memory crashes that could destabilize the service.

To resolve this, we utilized OS-level subprocess isolation. The actual ML pipeline is completely decoupled from the worker's business logic and executed via `subprocess.run()` with `capture_output=True`. The training script (`pipelines/first_ml_baseline/train.py`) runs as a separate OS process. Once the script finishes (or crashes), the OS reclaims all allocated memory. The worker process remains stable and insulated from failures in the ML code. The subprocess exit code is checked: a non-zero return triggers a `failed` status transition.

2.3.3. Low-Latency Inference Serving

The Inference Service is a FastAPI web service that receives JSON payloads through a Pydantic-validated `InputData` model and returns structured predictions via a `PredictResponse` schema.

Model bytes are fetched from MinIO with retry logic and cached in-process using `cachetools.LRUCache` (maxsize=10). Each prediction request is executed within a `ThreadPoolExecutor` (16 concurrent threads) via `asyncio.run_in_executor`, keeping the FastAPI event loop responsive. Inside each thread, the model is deserialized from bytes using `joblib.load()` with a per-thread

`functools.lru_cache` (maxsize=10), ensuring that repeated predictions on the same model do not re-deserialize.

Additionally, a Circuit Breaker pattern protects against MinIO outages: after 5 consecutive failures, the circuit opens and requests fail-fast with a 503 response. The circuit transitions to half-open after 30 seconds, allowing probe requests to test recovery. Three consecutive successes close the circuit again. Each request is tagged with a correlation ID via `asgi-correlation-id` middleware for distributed tracing.

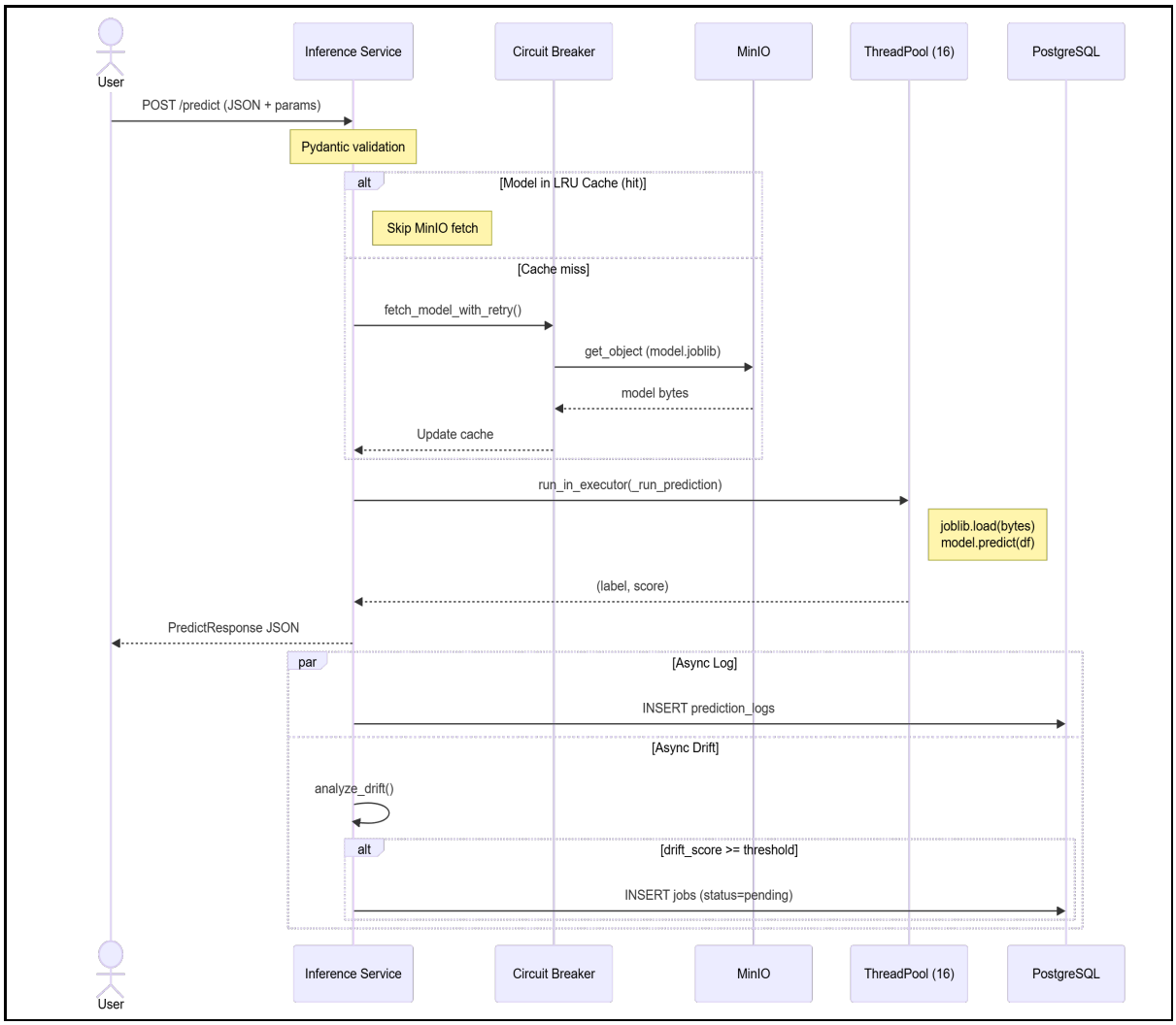

Figure 3: Inference + Drift Flow

Figure 3 illustrates the end-to-end inference request path, including model fetching with Circuit Breaker protection, threaded prediction execution, asynchronous prediction logging, and background drift evaluation that can trigger automatic retraining.

2.4. Storage, Telemetry & Observability

2.4.1. Artifact & Metric Repositories

A production platform requires logical separation between raw data, compiled artifacts, and telemetric metadata.

- **MinIO Object Storage:** Adopted as an S3-compatible blob layer with two dedicated buckets: `datasets` (raw CSV uploads) and `models` (serialized `.joblib` artifacts). File upload to MinIO uses `fput_object` (file-based), while model download during inference uses `get_object` (stream-based byte reading). This provides logical separation without incurring public cloud costs. 4
- **MLflow Experiment Registry:** Integrated to track algorithm metrics (Accuracy, Precision, Recall, F1) and hyperparameters. Each MLflow run is tagged with the originating database `job_id` via `mlflow.set_tag("job_id", job_id)`. After training completes, the Worker queries MLflow for runs matching this tag to retrieve metrics and parameters, linking the experiment record to the specific database job. 5

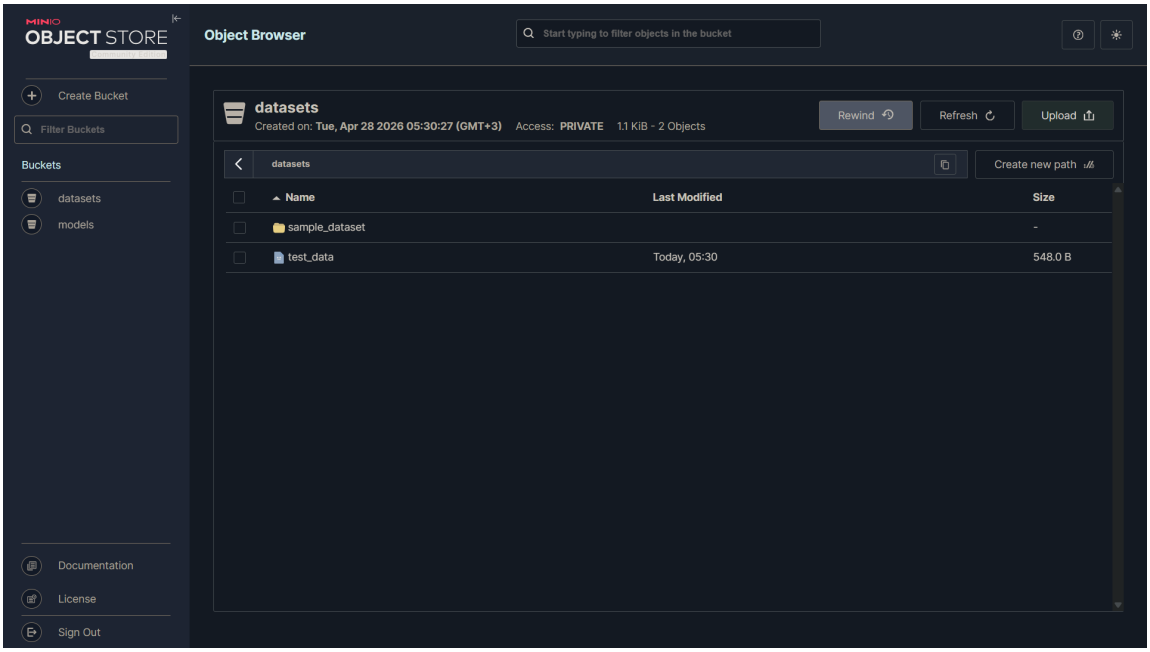

Figure 4: MinIO object storage console showing the two dedicated buckets: `datasets` (raw CSV uploads) and `models` (serialized `.joblib` artifacts).

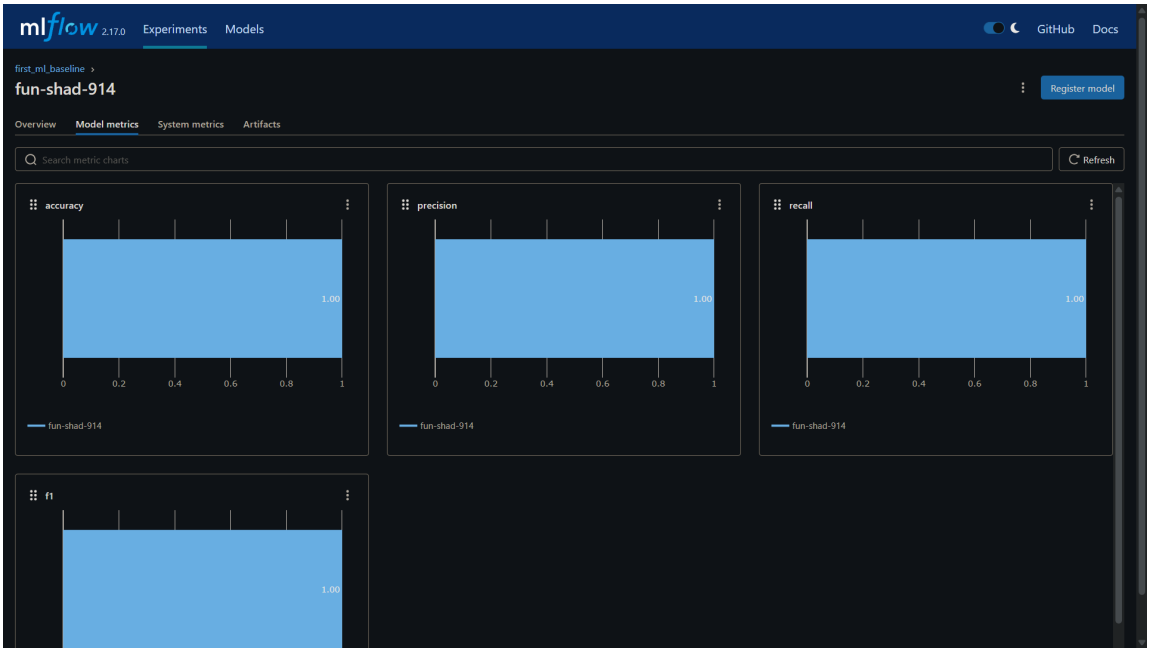

Figure 5: MLflow experiment tracking interface displaying logged metrics (Accuracy, Precision, Recall, F1), hyperparameters, and the `job_id` tag used to link each run to the corresponding database job.

2.4.2. System Monitoring Dashboard

The Monitoring Dashboard is a Streamlit application providing read-only visibility across all platform subsystems. It connects directly to the PostgreSQL data layer and comprises eight pages: System Overview, Jobs, Datasets, Models, Deployments, Inference, Monitoring, and System Health. The dashboard includes mock authentication and a sidebar-based navigation system. This approach was chosen for rapid Python-native UI development without the overhead of a dedicated JavaScript frontend.

Figure 6: Monitoring Dashboard – System Overview page. Provides an at-a-glance summary of jobs, models, deployments, and recent platform activity.

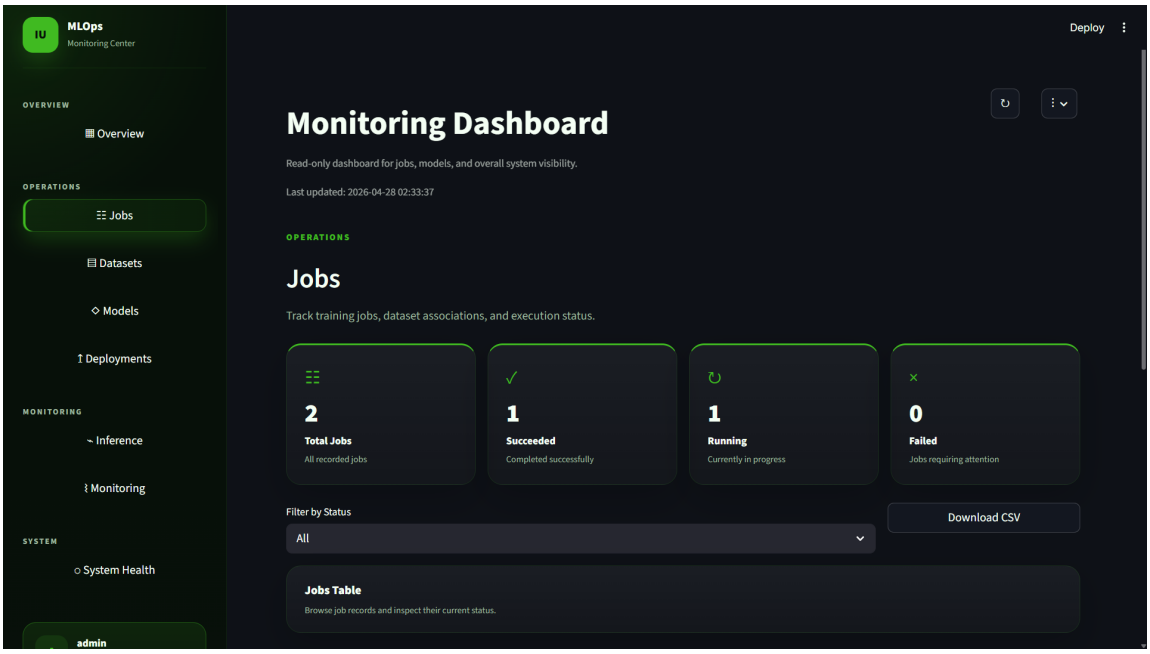

Figure 7: Monitoring Dashboard – Jobs page. Displays all training jobs with their current lifecycle status (`total`, `running`, `succeeded`, `failed`) and associated dataset metadata.

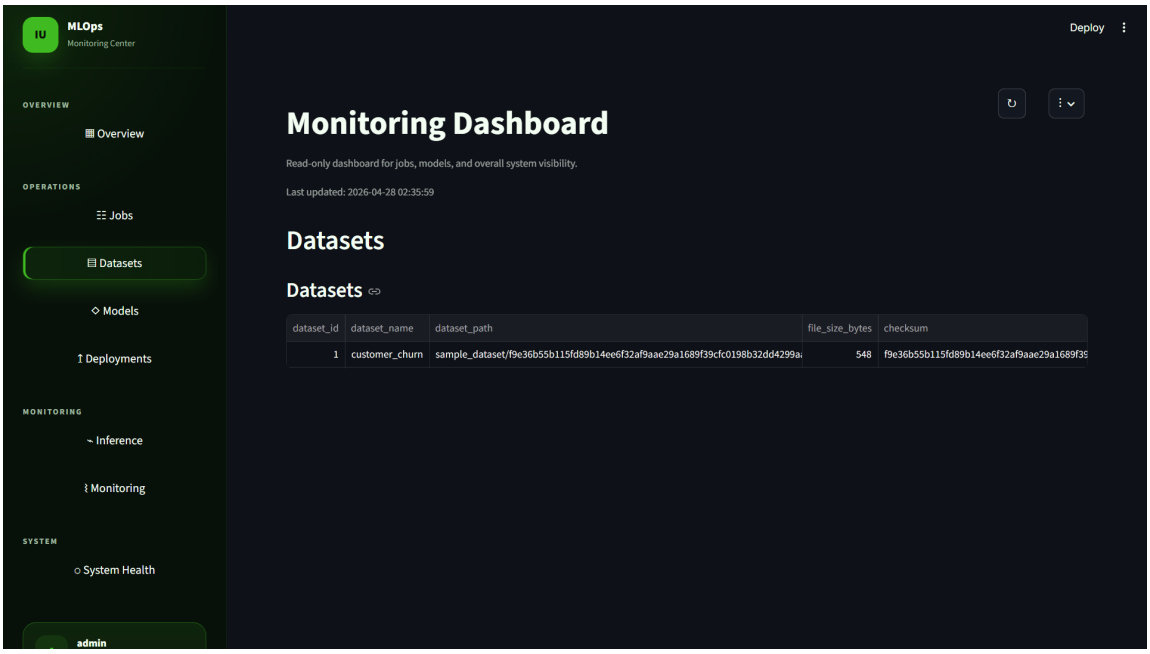

Figure 8: Monitoring Dashboard – Datasets page. Lists all registered datasets with their storage paths in MinIO, file sizes, and format metadata.

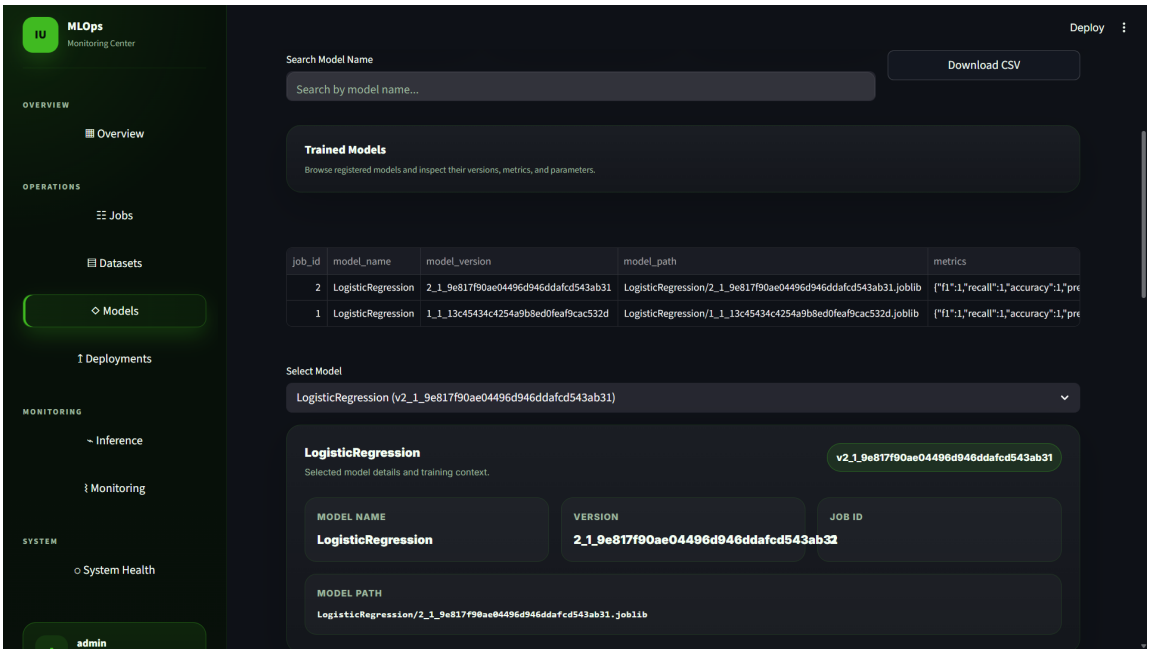

Figure 9: Monitoring Dashboard – Models page. Shows trained models with their versions, MLflow metrics (Accuracy, Precision, Recall, F1), source job IDs, and MinIO artifact paths.

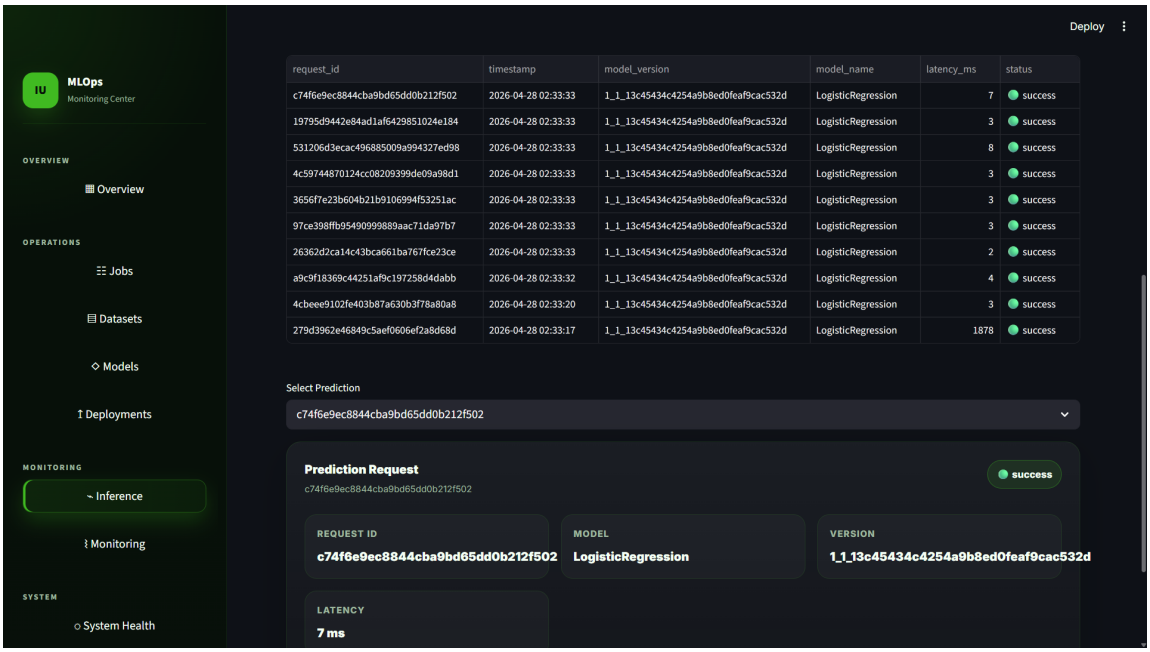

Figure 10: Monitoring Dashboard – Inference page. Displays prediction history including per-request latency, model versions used, and prediction outcomes.

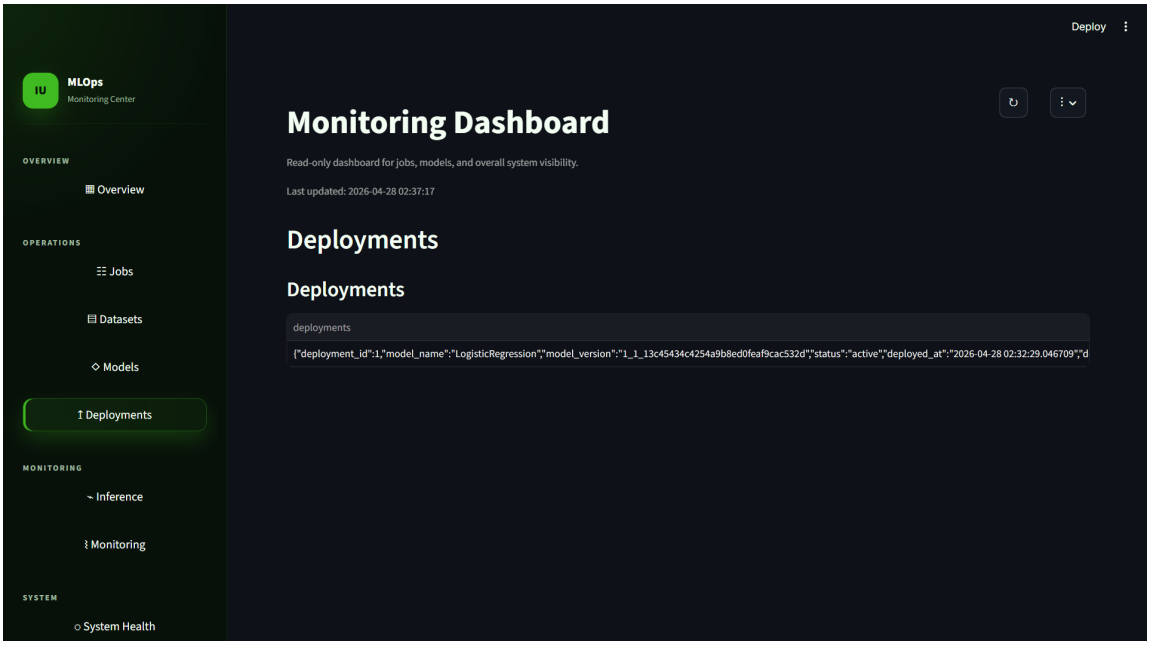

Figure 11: Monitoring Dashboard – Deployments page. Lists active model deployments with their versions, deployment timestamps, and the user or system that triggered the promotion.

2.5. Robustness, Security & Quality Assurance

2.5.1. Quality Control & Code Standards

To maintain structural integrity across a distributed development team, strict CI/CD mechanisms and pre-commit hooks are enforced:

- **Black, Flake8 & isort:** Ensures deterministic code formatting, static analysis compliance, and consistent import ordering. Enforced via pre-commit hooks and CI pipeline.
- **detect-secrets:** Scans all commits to prevent hardcoded API keys or database credentials from entering the repository, managed via a strict baseline file (`.secrets.baseline`).
- **Commitizen:** Enforces Conventional Commits format on all commit messages.
- **PyTest Framework:** Unit tests (`tests/unit/`), service-level tests (`tests/services_tests/`), and integration tests (`tests/integration/`) validating pipeline idempotency, DB transactions, API contracts, worker crash recovery, concurrency safety, timeout handling, and drift detection.
- **GitHub Actions CI:** Automated pipeline running lint, tests, Docker build verification for all services, and Trivy vulnerability scanning on every push and pull request.

2.5.2. Network Security & Data Validation

Inbound data for prediction requests is validated via Pydantic schemas (`InputData`, `Prediction`, `PredictResponse`). Malformed payloads are rejected with **422 Unprocessable Entity** before reaching application logic. The Inference Service additionally returns 404 when a requested model is not found, 503 when MinIO is unavailable (via Circuit Breaker), and 500 for unexpected errors. Dataset uploads to `/datasets` are validated for file format (CSV only) and non-empty payload before processing.

3. Results & Discussion

3.1. Operational Standards & System Constraints

The platform's architectural blueprints dictate several rigorous operational constraints necessary for production deployment.

3.1.1. Resource Allocation & Hardware Acceleration

To prevent single-component resource starvation, hardware constraints are strictly enforced at the container level via cgroups (defining rigid RAM and CPU limits). Furthermore, the execution architecture is designed to accommodate advanced mathematical hardware; the `subprocess` isolation layer inherently provisions the integration of GPU-accelerated computing required for future deep learning pipelines.

3.1.2. Fault Tolerance & Graceful Degradation

- **Ephemeral Workspace:** The Training Worker stores downloaded datasets and training artifacts in a configurable temporary directory (`WORKER_TMP_DIR`, defaulting to the system temp directory). Automated cleanup of these files after job finalization is planned for a future iteration.
- **Stuck Job Recovery:** If the Training Worker process is terminated (e.g., SIGKILL) while a job is in the `running` state, the next Worker startup executes a single SQL query that resets all `running` jobs back to `pending`. This recovery runs once during the FastAPI lifespan initialization, before the polling loop begins, ensuring no jobs remain permanently stuck.

3.1.3. Telemetry & Observability Standards

Comprehensive observability is a hard architectural requirement imposed by the platform's design tradeoffs. The platform runs training jobs inside isolated OS subprocesses, communicates through database state mutations rather than direct RPC, and triggers automated retraining based on drift scores. These decisions decouple services and prevent cascading failures, but they also make it impossible to debug a pipeline by following a single log stream – the operator must correlate state changes across PostgreSQL, MinIO, MLflow, and stdout logs from independent containers. To mitigate this, the telemetry system operates on two levels. First, a custom `StructuredLogger` (`shared/logging_utils.py`) outputs structured JSON with fields: `timestamp`, `level`, `service`, `event`, `module`, `function`, `line`, and arbitrary key-value context, enabling ingestion by external aggregation systems (e.g., ELK stack, Loki). However, not all services consistently use this logger – the Training Worker falls back to plain-text `logging.basicConfig` in several modules, producing unstructured output that breaks automated log parsing. Second, the Inference Service assigns per-request correlation IDs via `asgi-correlation-id` middleware and logs every prediction to the `prediction_logs` table, but this tracing does not propagate to other services – the Orchestrator and Training Worker do not read or emit `X-Correlation-ID`, so end-to-end request tracing from dataset upload through training to inference is not possible. Additionally, the `prediction_logs` table has no retention policy or TTL, meaning it grows unboundedly as inference traffic increases. These gaps are acknowledged limitations of the current implementation and are targeted for resolution in **Phase 2**.

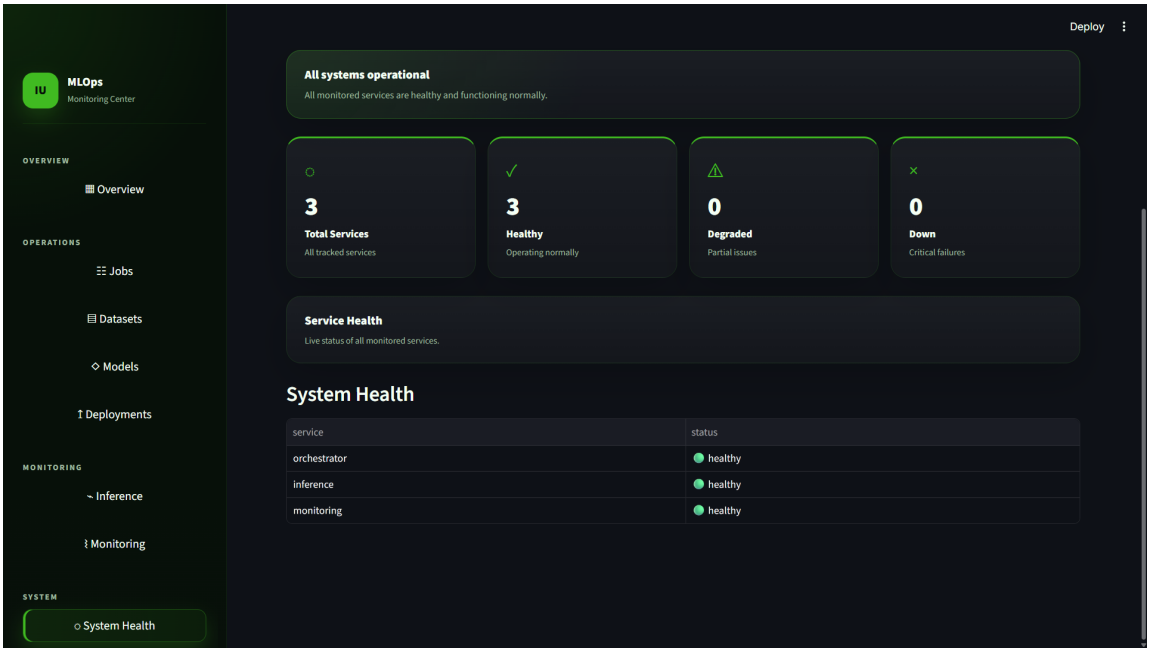

Figure 12: Monitoring Dashboard – System Health page. Shows service availability status.

3.2. Development Roadmap & Future Work

The Distributed MLOps Platform is developed using an iterative, phase-based methodology. Phase 1 (Foundation & Core Execution) has been successfully validated, establishing the core microservices and asynchronous database-driven workflows. Subsequent phases are designed to elevate the platform to enterprise-grade maturity, focusing on hardware acceleration, strict data lineage, and advanced observability.

3.2.1. Current Capabilities (Phase 1: Foundation)

- **Microservice Orchestration:** Fully containerized deployment via Docker Compose with cgroup resource boundaries (memory/CPU limits and reservations per service).
- **Asynchronous Execution:** Fault-tolerant job queueing using PostgreSQL `FOR UPDATE SKIP LOCKED` and OS-level subprocess isolation with exit-code-based failure detection.
- **Concurrent Inference:** Thread-based prediction execution (`ThreadPoolExecutor`, 16 workers) with in-process model caching and per-thread deserialization caching.
- **Artifact Management:** S3-compatible storage (MinIO) with separate buckets for datasets and models, and experiment tracking via MLflow.
- **Drift Detection:** Per-prediction statistical drift analysis against a reference profile, with automatic retraining job creation when the drift score exceeds a configurable threshold (default: 2.0).
- **Model Lifecycle:** Dataset registration (`POST /datasets`), training job creation (`POST /train`), model promotion (`POST /promote`), and deployment listing (`GET /deployments`).
- **Observability:** Streamlit monitoring dashboard, structured JSON logging, prediction logging to PostgreSQL, correlation ID tracing, and Grafana dashboard definitions (provisioned JSON configurations ready for deployment, but not yet wired into the Docker Compose stack).

- **Resilience:** Circuit Breaker pattern for MinIO connectivity, DB connection retries on startup (30 attempts), MinIO download retries (3 attempts), and stuck job recovery on worker startup.
- **CI/CD:** GitHub Actions pipeline with lint (Black, Flake8, isort), unit and service-level tests, Docker build verification, Trivy security scanning, commitizen commit formatting, and `detect-secrets`.

3.2.2. Target Milestones (Phase 2 & 3: Scaling & Reliability)

- **Hardware Acceleration & GPU Support:** Expanding the execution plane to support CUDA-enabled training nodes for deep learning pipelines.
- **Data Lineage (DVC):** Full integration of Data Version Control pipelines to programmatically track dataset state transitions and reproducibility.
- **Advanced Observability:** Transitioning to a dedicated JavaScript-based frontend dashboard with advanced data-density visualizations, replacing the Streamlit baseline.
- **Ephemeral Cleanup:** Implementing automated purging of temporary training artifacts after successful job finalization.
- **Periodic Retraining:** Extending the Scheduled Retraining Service (currently standalone) to be fully integrated into the Docker Compose deployment with configurable intervals.

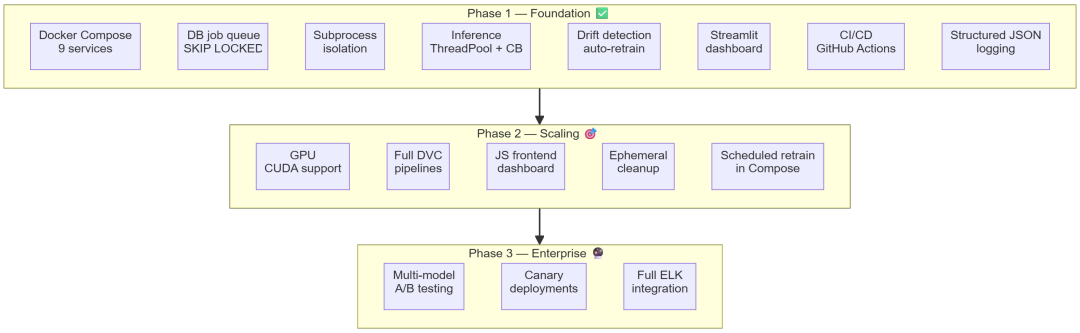

Figure 13: Phased Development Roadmap illustrating current foundational achievements and targeted architectural milestones.

3.2.3. Team Contributions

The project was developed collaboratively over three weeks by a team of eight engineers, with each member responsible for a defined subsystem. The following table summarizes the primary areas of ownership.

Table 1: Team member responsibilities and subsystem ownership.

Member	**Primary Focus**
Andrei P. (Team Lead)	Integration, CI/CD pipeline, testing infrastructure, documentation, overall architecture
Arina S. (Project Manager)	Issue tracking, coordination, team communication
Elizaveta K.	Orchestrator service, database schema design, dataset endpoints
Ahmed L.	Monitoring dashboard (full implementation: pages, authentication, navigation), demo creation
Aleksei	Inference service, circuit breaker, resource cleanup
Danil K.	Training worker, MLflow integration, subprocess isolation
Daria G.	Baseline training pipelines, dataset registration flow
Julia K.	Technical report and presentation

4. References & Official Specifications

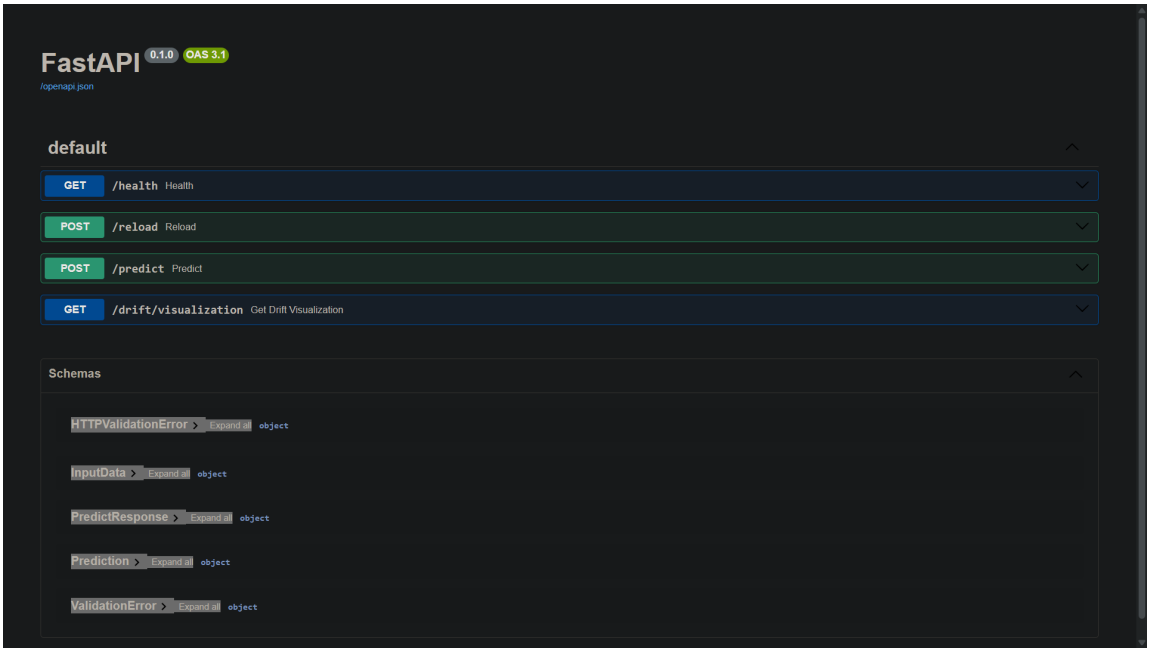

Figure 14: Platform API documentation (FastAPI auto-generated at `/docs`). Covers all Orchestrator and Inference Service endpoints with request/response schemas.

4.1. References

1. Sculley, D., et al. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems* (Vol. 28, pp. 2503–2511).
2. Huyen, C. (2022). *Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications*. O'Reilly Media.
3. Python Software Foundation. (n.d.). *concurrent.futures – Launching parallel tasks*. Python 3.12 Documentation. https://python.org
4. PostgreSQL Global Development Group. (n.d.). *The SELECT Command: FOR UPDATE SKIP LOCKED*. PostgreSQL Documentation. https://postgresql.org
5. Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.